

Splay Tree

→ Splay tree is a self adjusting tree

- Don't balance
 - Strict balance
 - Pretty good balance
 - Adjust on access
- ①

* Ordinary binary search tree has no balance condition

* Balanced tree like AVL trees enforce a balance condition when nodes change

→ tree is always balanced after insert or delete.

→ Self adjusting trees get reorganized over time as nodes are accessed.

* Tree adjusts after insert, delete, or find.

* Splay trees are tree structures that:

* Are not perfectly balanced all the time

* Data most recently accessed is near the root.

(principle of locality of reference, 80-20 rule)

* The Procedure:

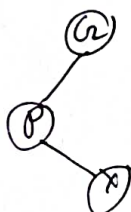
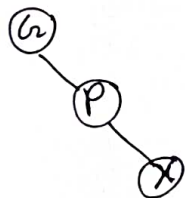
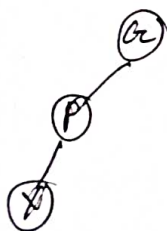
* After node x is accessed, perform "splaying" operations to bring x to the root of the tree.

* Do this in a way that leaves the tree more balanced as a whole.

Let x be a non-root node with ≥ 2 ancestors.

→ P is the parent node

→ G is the grand parent node.

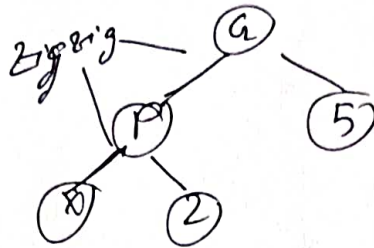


Splay Tree

(2)

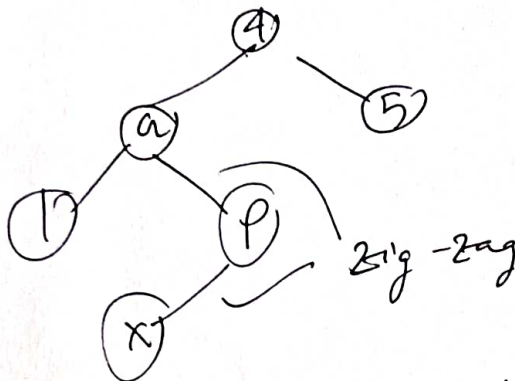
Parent and Grand parent in same direction.

Zig-Zig

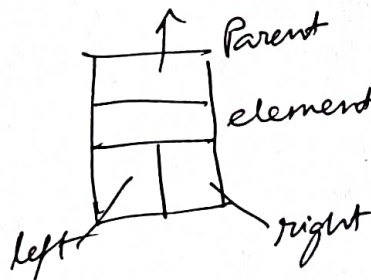


Parent and Grandparent in different directions

Zig-Zag



A node structure:



When X is accessed, apply one of right rotation routines

→ Single Rotation (X has a P (the root) but no G)

ZigFromLeft, ZigFromRight

→ Double Rotations (X has both a P and a G)

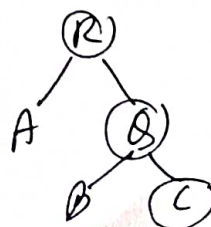
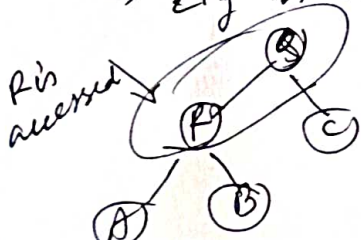
ZigZigFromLeft, ZigZigFromRight

ZigZagFromLeft, ZigZagFromRight

Zig at depth 1 (root)

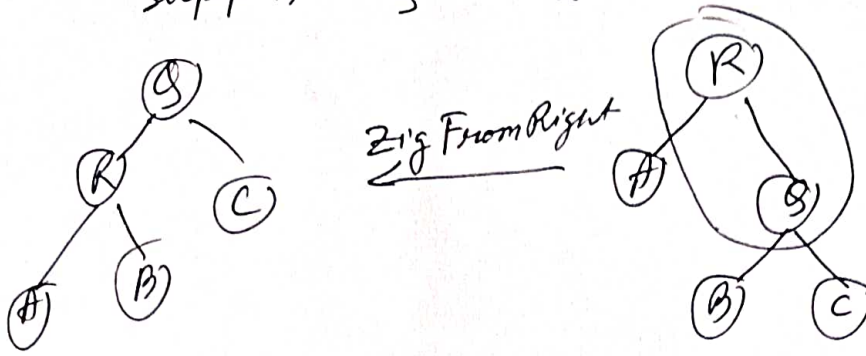
→ Zig is a single rotation

Zig fromLeft



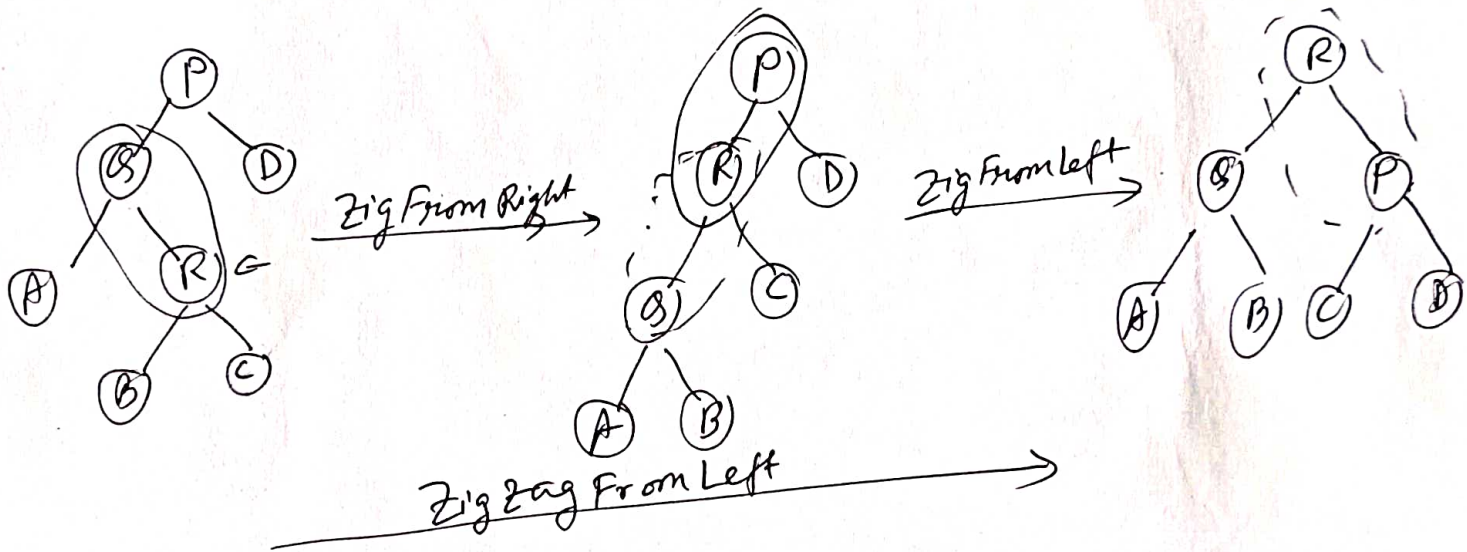
Splay Tree

→ From the previous example.
Suppose Q is now accessed using Find.



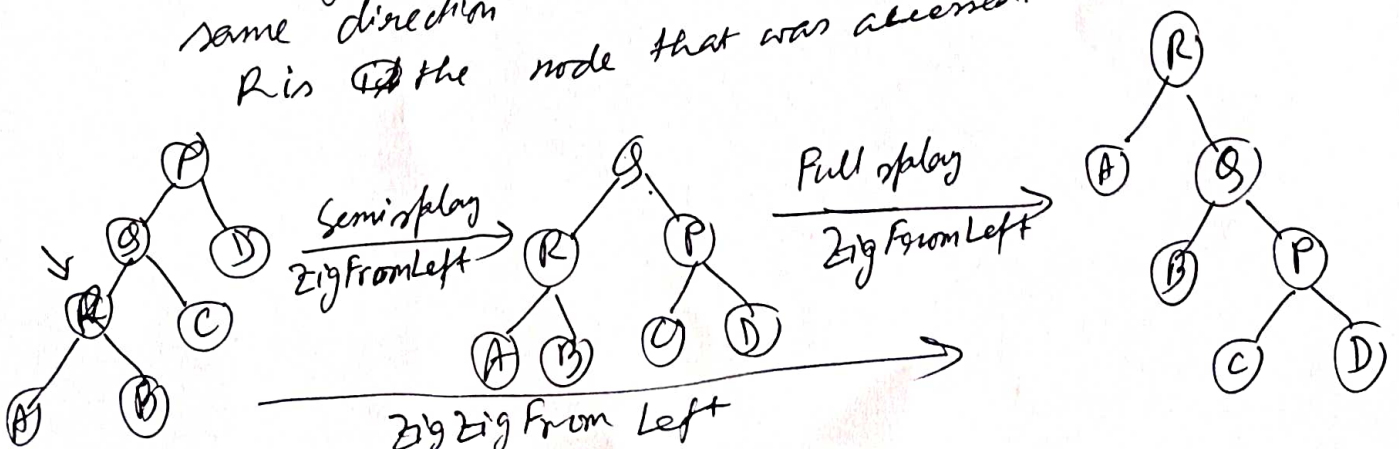
Zig Zag Operation

* Zig Zag consists of two rotations of the opposite direction
Here, assume R is the node that was accessed.



Zig Zig Operation

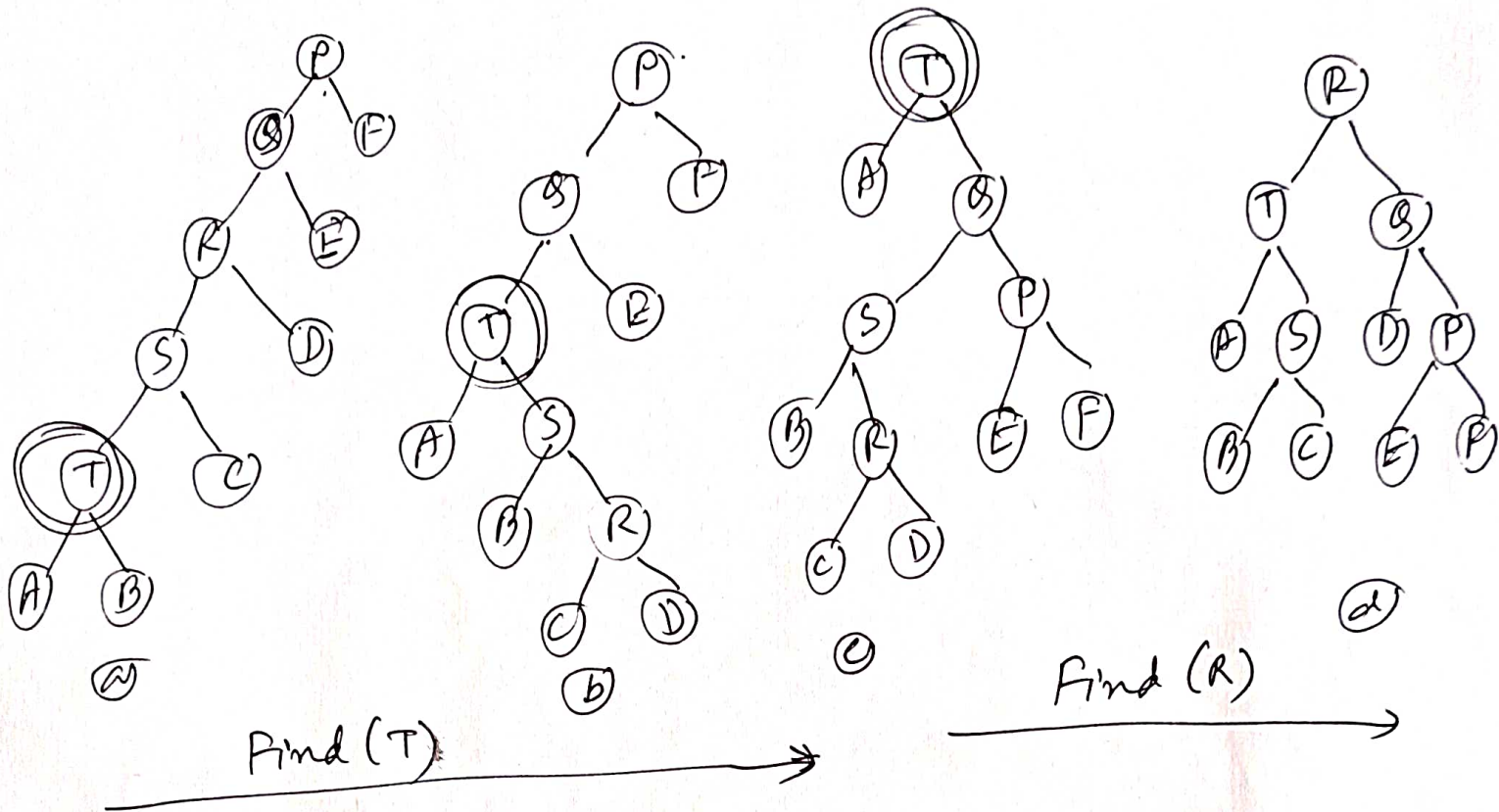
* Zig Zig operation consists of two single rotations of the same direction
R is the node that was accessed.



Splay Tree

(4)

Decreasing depth - "auto balance".



Splay Tree Insert and Delete

* Insert x
 - Insert x as normal then splay x to root

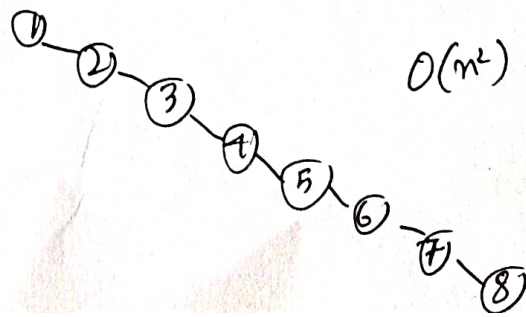
* Delete x
 - Splay x to root and remove it $\rightarrow$ Two tree remain, right subtree and left subtree.

- Splay the max in the left subtree to the root.

- Attach the right subtree to the new root of the left subtree.

Insert Example

* Inserting in order 1, 2, 3, ... 8
 * Without self-adjustment

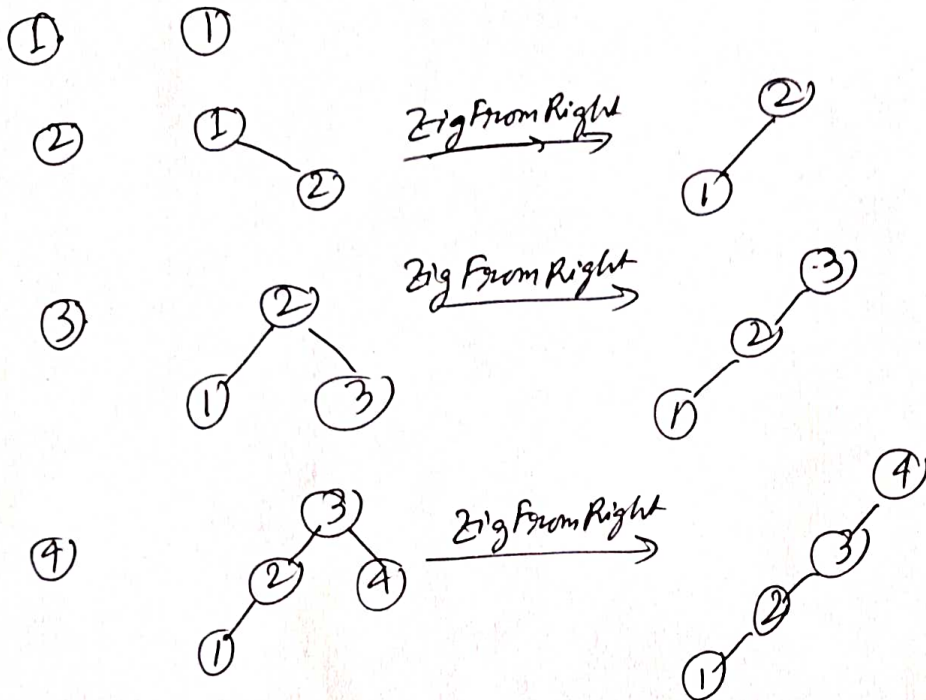


$O(n^2)$ time for n insert.

Splay Tree

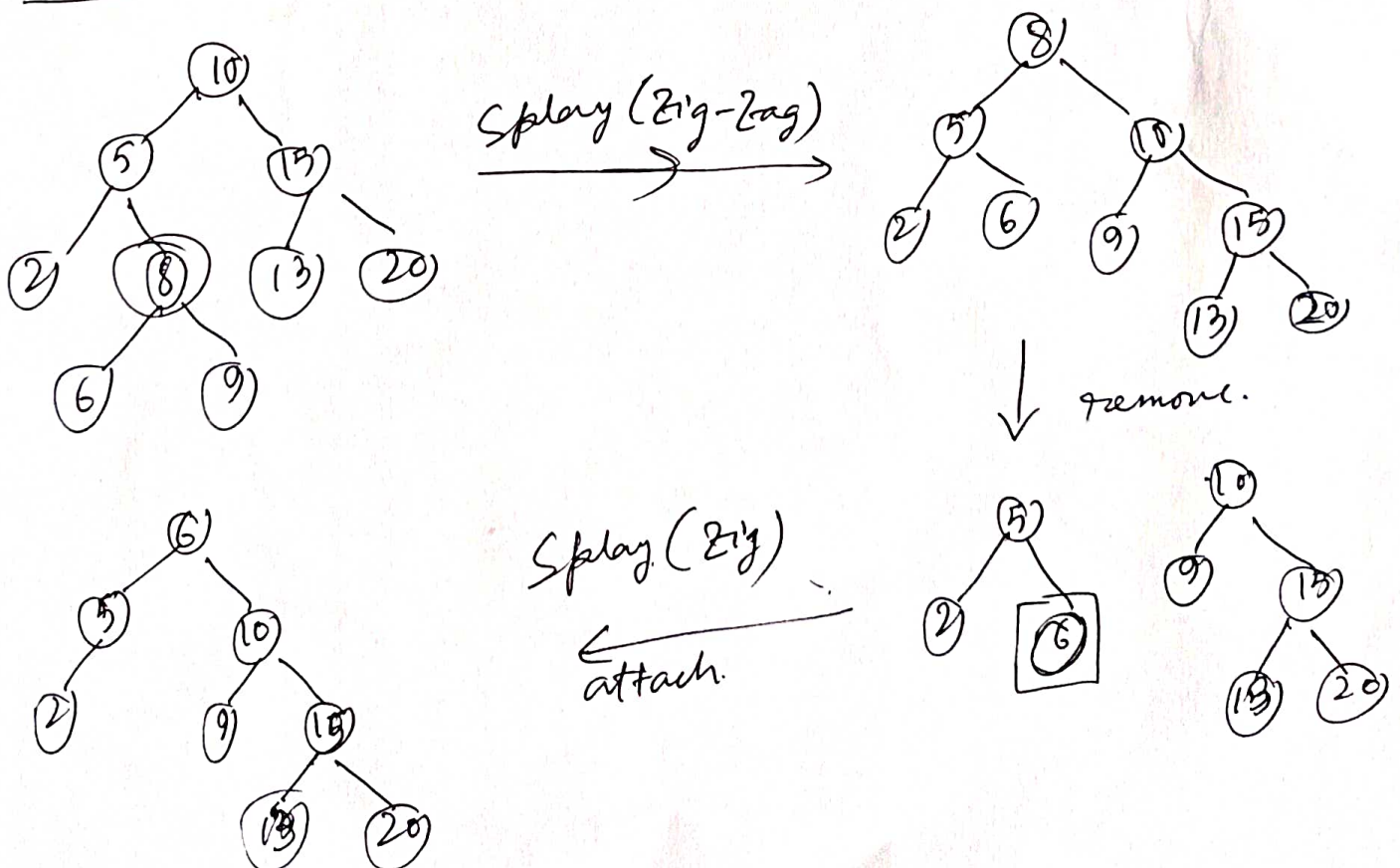
(5)

& Insert with Self-Adjustment.



Each Insert takes $O(1)$ time therefore $O(n)$ time for n Insert!

Deletion Example



Splay Tree

(6)

Analysis of Splay Tree

- * Splay trees tend to be balanced.
 - M operations takes time $O(M \log N)$ for $M \gg N$
operations on N items
 - Amortized $O(\log M)$ time
- * Splay trees have good "locality" properties.
 - Recently accessed items are near the root of the tree
 - Items near an accessed one are pulled toward the root.